UTD Secure HSM Design

MITRE eCTF 2026

University of Texas at Dallas

eCTF Team

February 2026

Abstract. This document describes the security architecture for UT Dallas's submission to the 2026 MITRE Embedded Capture the Flag (eCTF) competition. The system implements a permissioned Hardware Security Module (HSM) for securely storing and transferring IC fabrication data. The design uses AES-256-GCM authenticated encryption via the MSPM0L2228 AESADV hardware peripheral for data at rest and in transit, HMAC-SHA256 (wolfcrypt) for mutual authentication and permission verification, hardware-TRNG-derived nonces and challenges, and a suite of countermeasures against fault injection and side-channel analysis.

Contents

1 System Overview **2**
1.1 Competition Context . . . 2

2 Security Requirements Compliance **2**

3 Threat Model **3**
3.1 Attack Goals . . . 3

4 Cryptographic Architecture **3**
4.1 Key Split . . . 3
4.2 Algorithms . . . 4

5 Data Structures **4**
5.1 Stored File (`file_t`) . . . 4
5.2 AAD Formats . . . 4
5.3 Wire Permission Format . . . 5

6 Protocol Diagrams **5**
6.1 PIN Verification . . . 5
6.2 RECEIVE Protocol (Mutual Authentication) . . . 5
6.3 INTERROGATE Protocol . . . 7

7 Command Reference **7**
7.1 READ Flow . . . 8

8 Security Countermeasures **8**
8.1 PIN Protection . . . 8
8.2 Permission Enforcement . . . 9
8.3 Side-Channel Attack Mitigations . . . 9
8.4 Memory Safety . . . 9
8.5 Stack Canary . . . 10

9 Attack Mitigation Summary **11**

1 System Overview

1.1 Competition Context

The 2026 eCTF scenario has teams design a Hardware Security Module (HSM) for ChipCorp's confidential IC fabrication workflow. Each deployed set of HSMs consists of three roles:

- **Engineer HSM** — Holds design files; has full read/write/receive permissions for the design permission group. PIN unknown to attacker.
- **Technician HSM** — Holds firmware update and calibration data; no permission to read or receive design files. PIN known to attacker.
- **Photolithography Machine HSM** — Receives files from both other HSMs; attacker has remote-only transfer-interface access.

Each HSM is built from the same global secrets file but with distinct permissions and PIN, yielding unique `PERMISSION_MAC` and `PIN_HMAC` values per device. `PIN_KEY` is generated independently per HSM inside `secrets_to_c_header.py` so that a CPA recovery of one device's PIN key does not affect any other device.

2 Security Requirements Compliance

SR	Requirement	Implementation
SR1	No file action without a validly provisioned HSM with the correct permission for that group	HMAC challenge-response; `PERMISSION_MAC`; `validate_permission()` double-pass
SR2	No PIN-protected action completable without prior knowledge of the PIN; PIN must not be exposed	`SECURE_PIN_CHECK` at every management-interface handler entry; PIN never stored in flash
SR3	HSM must not receive any file not generated by a valid HSM with write permission for that group	4-round mutual HMAC handshake; GCM tag over Transfer AAD binding challenges, slot, UUID, group, and name
SR4	File integrity must be cryptographically protected	AES-256-GCM tag; AAD binds slot, UUID, group, and name at rest and in transit
SR5	Files must be confidential at rest and in transit	`STORAGE_KEY` (at rest); `TRANSFER_KEY` (wire); independent 256-bit keys

3 Threat Model

The attacker has full physical access, a ChipWhisperer power-analysis platform, and the complete source code for the Technician and Engineer HSMs. The attacker knows the Technician PIN (`1a2b3c`) but not the Engineer PIN. The Photolithography Machine is accessible only via its UART1 transfer interface; no physical access or management interface is available to the attacker.

3.1 Attack Goals

- **Read Design** — Read `design_file1` from Engineer HSM without the Engineer PIN.
- **Steal Design** — Cause Engineer HSM to transfer `design_file1` to a device without Design receive permission.
- **Read Update** — Read `update_file` from Technician HSM without read permission on the Update group.
- **Compromise Machine** — Cause Photolithography HSM to accept a corrupted update file via the transfer interface.
- **Backdoored Design** — Cause Photolithography HSM to accept a specially crafted design file via the transfer interface.

4 Cryptographic Architecture

4.1 Key Split

`gen_secrets.py` generates five global 256-bit keys using Python's `secrets.token_bytes(32)` (CSPRNG backed by `/dev/urandom`). `CHALLENGE_TWEAK` is a 12-byte deployment-wide constant from the same source. `PIN_KEY` is generated independently per device inside `secrets_to_c_header.py` using `os.urandom(32)`.

Key	Size	Scope	Purpose
`STORAGE_KEY`	256 bit	Global	AES-256-GCM for files at rest
`TRANSFER_KEY`	256 bit	Global	AES-256-GCM for files in transit
`TRANSFER_AUTH_KEY`	256 bit	Global	HMAC-SHA256 for protocol challenge-response only
`PERM_MAC_KEY`	256 bit	Global	HMAC-SHA256 for `PERMISSION_MAC` only (build-time; never enters runtime hardware)
`CHALLENGE_TWEAK`	12 byte	Global	XOR mask applied to every handshake nonce before HMAC; breaks chosen-input CPA on `TRANSFER_AUTH_KEY`
`PIN_KEY`	256 bit	Per-device	HMAC-SHA256 for PIN verification; independent per HSM

Rationale. Separating storage and transfer keys means a CPA campaign against AES during READ reveals only `STORAGE_KEY`, giving no information about `TRANSFER_KEY` or any HMAC key. `PERM_MAC_KEY` is architecturally isolated from `TRANSFER_AUTH_KEY`: an attacker performing CPA on handshake traces can at most recover `TRANSFER_AUTH_KEY`, which does not let them forge a `PERMISSION_MAC` (signed with the build-time-only `PERM_MAC_KEY`). `CHALLENGE_TWEAK` XOR'd into every nonce before the HMAC call removes the attacker's ability to choose the actual HMAC first-block input even when they control the wire nonce, eliminating the chosen-input advantage required for standard CPA. `PIN_KEY` is per-device so recovering one device's PIN key reveals nothing about any other device.

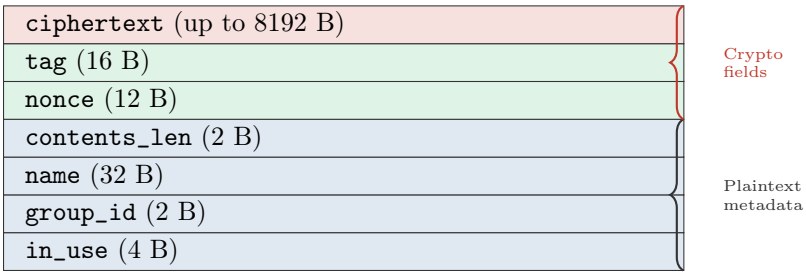
4.2 Algorithms

- **Symmetric encryption + integrity:** AES-256-GCM via the MSPM0L2228 `AESADV` hardware peripheral in `DL_AESADV_MODE_GCM_AUTONOMOUS`. The hardware derives GHASH subkey H and computes Y_0-encrypted internally; no separate ECB pass is needed. The peripheral is reset after every call to clear key registers. CPU-polling mode (DMA disabled); data fed in 16-byte aligned blocks.
- **MAC / authentication:** HMAC-SHA256 via wolfcrypt (`wc_HmacSetKey`, `wc_HmacUpdate`, `wc_HmacFinal`). The MSPM0L2228 has no hardware HMAC accelerator.
- **Nonce generation:** Hardware TRNG (`DL_TRNG`) producing 12-byte nonces per operation. Nonce reuse probability: $\approx 2^{-96}$.
- **Domain separation:** All HMAC calls include a mandatory, null-terminated domain label appended to the data (e.g., `"pin"`, `"sender"`, `"interrogate_req"`). No domain-free HMAC variant exists.

5 Data Structures

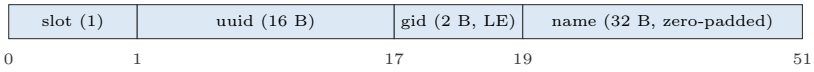
5.1 Stored File (`file_t`)

Files at rest in flash have the following layout. Only `ciphertext` is encrypted; the header fields are integrity-protected via the GCM AAD and tag.



5.2 AAD Formats

Storage AAD (51 bytes) binds ciphertext to its slot, UUID, group, and filename:



Transfer AAD (75 bytes) additionally binds both challenges, slot, and UUID:

recv_chal (12)	send_chal (12)	sl	uuid (16)	gid (2)	name (32 B)

0 12 24 25 41 43 75

5.3 Wire Permission Format

Permissions are serialized as 5 bytes per entry: `group_id[2 LE] || read[1] || write[1] || receive[1]`. Boolean fields must be exactly `0x00` or `0x01` (enforced by `validate_bool()`).

6 Protocol Diagrams

6.1 PIN Verification

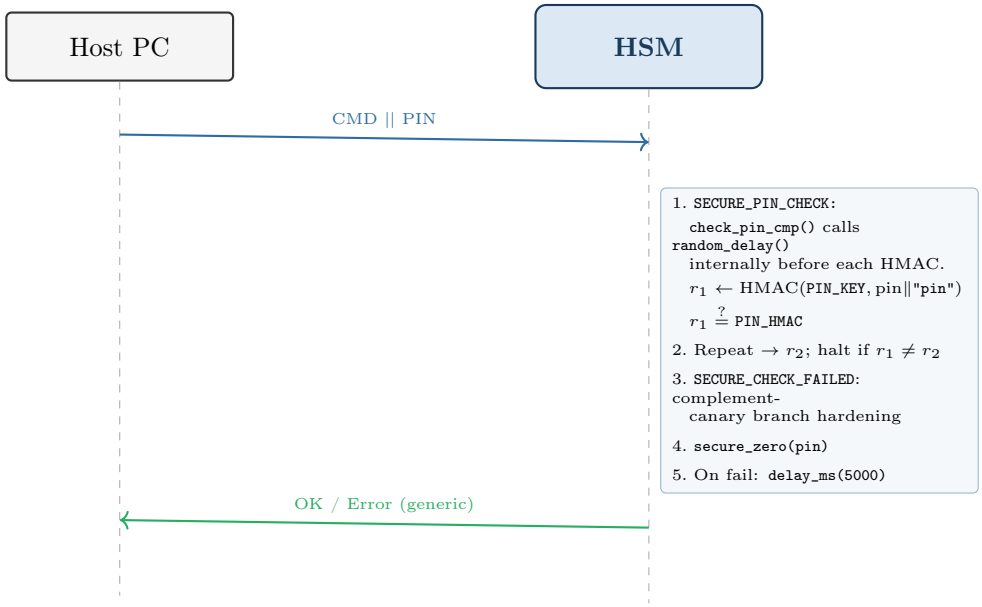


Figure 1: PIN verification flow. `random_delay()` fires inside each `check_pin_cmp()` call, so both passes have independent jitter. `SECURE_CHECK_FAILED` uses complement-canary hardening to resist a single branch-skip glitch after the double-pass.

6.2 RECEIVE Protocol (Mutual Authentication)

The RECEIVE command performs a 4-round mutual authentication handshake using HMAC-SHA256 and then transfers the file under AES-256-GCM with `TRANSFER_KEY`. The *Initiator* is the requesting HSM; the *Responder* holds the file. All HMAC calls on both sides apply `CHALLENGE_TWEAK` (XOR with nonce) before the hash to defeat chosen-input CPA.

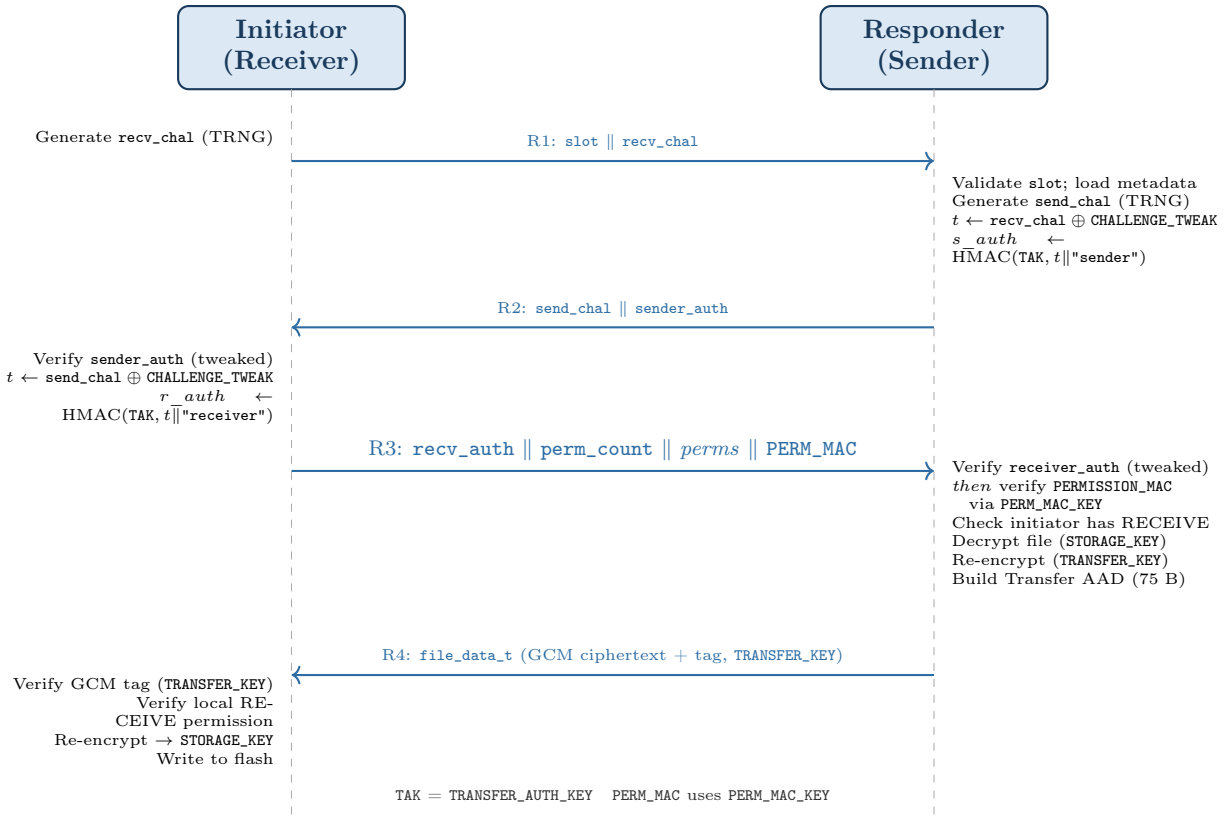


Figure 2: RECEIVE mutual authentication protocol. `TRANSFER_AUTH_KEY` is used only for challenge-response; `PERM_MAC_KEY` signs and verifies `PERMISSION_MAC`. `receiver_auth` is verified *before* `PERMISSION_MAC` so an unauthenticated sender cannot reach `PERM_MAC_KEY` operations with chosen permission bytes.

6.3 INTERROGATE Protocol

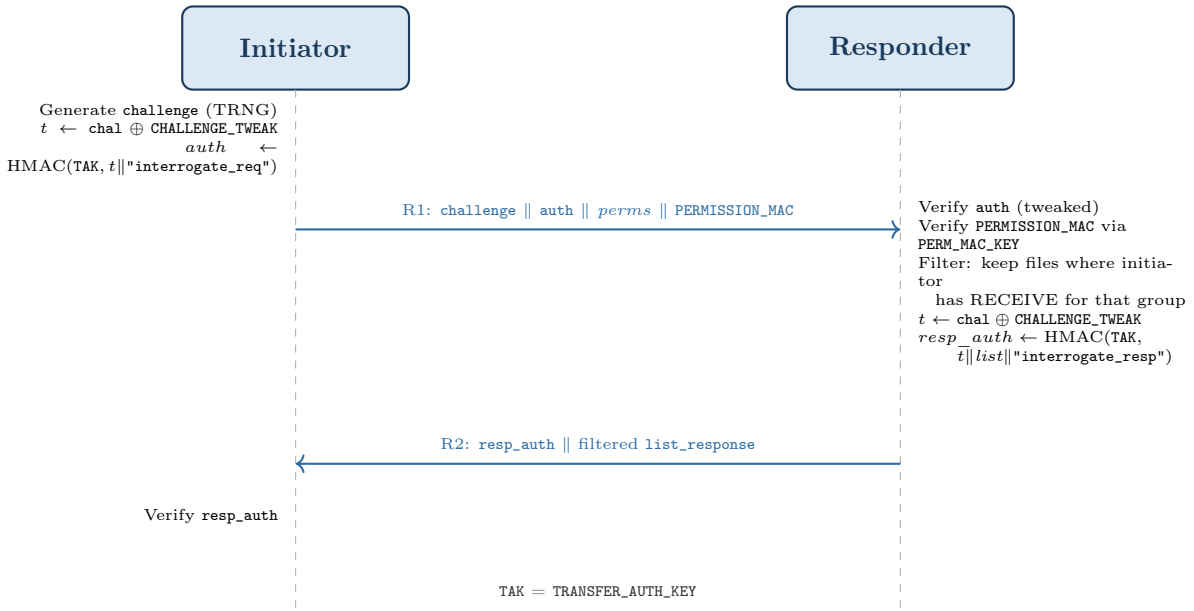


Figure 3: INTERROGATE protocol. The tweaked challenge binds the response, preventing a rogue responder from returning a cached or fabricated file list. `PERM_MAC_KEY` verifies permissions; `n_files` is bounds-checked before forwarding to the host.

7 Command Reference

All management-interface commands (LIST, READ, WRITE, RECEIVE, INTERROGATE) are served on UART0 and are PIN-protected. LISTEN is served on UART1 (transfer interface) and requires no PIN; it handles exactly one peer command per invocation. Messages arriving on the wrong interface are rejected with a generic error before any processing occurs.

7.1 READ Flow

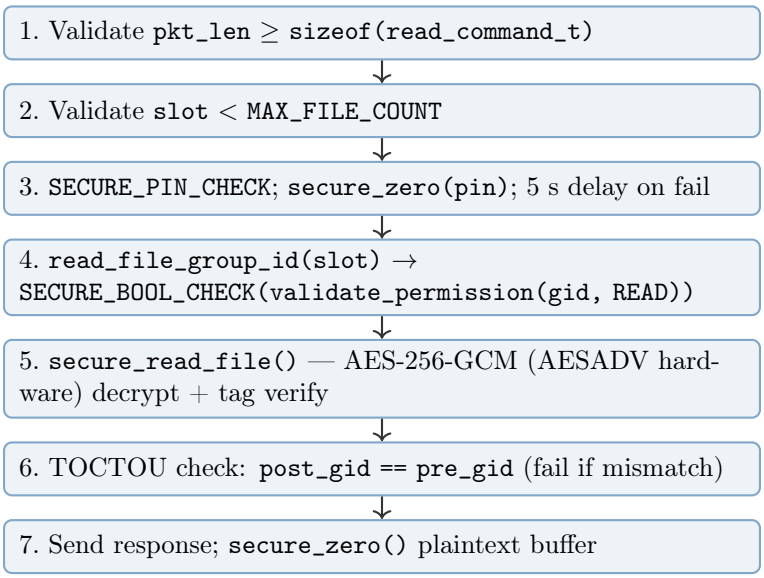


Figure 4: READ command flow. Permission is checked before decryption; a TOCTOU re-check ensures the group ID was not altered between reads. The WRITE flow is symmetric: PIN check → permission(WRITE) check → AES-256-GCM encrypt → store to flash.

8 Security Countermeasures

8.1 PIN Protection

- **HMAC-based storage.** The PIN is never stored in flash; only `PIN_HMAC` (see §4) is stored. Even if an attacker reads both from flash, offline brute-force requires $16^6 \approx 16$M HMAC evaluations without any on-device oracle.
- **Double-evaluation.** `SECURE_PIN_CHECK` calls `check_pin_cmp()` twice. If both results disagree, `security_halt()` fires immediately. A single fault injection cannot bypass both passes simultaneously.
- **Complement-canary branch hardening.** `SECURE_CHECK_FAILED` re-reads `ok1` and `ok2` from stack into separate `volatile` temporaries, computes their AND, and stores the bitwise complement as a canary. The branch passes only when the AND is true *and* the canary is its complement. A glitch corrupting one load or the AND cannot simultaneously corrupt the canary, so a single fault cannot silently accept a failed check.
- **Rate limiting.** A wrong-PIN result incurs a 5-second delay before the generic error is returned. Brute-force over the serial interface would take ≈ 2 years.
- **Immediate zeroing.** `secure_zero(pin)` is called at every check site before any other work proceeds.

8.2 Permission Enforcement

`validate_permission()` iterates `global_permissions[]` with a double-pass pattern: both passes must agree or the device halts. `SECURE_BOOL_CHECK` wraps every call site. On the transfer interface, the listening HSM re-verifies its own local permissions for the file's group after decryption (`PERM_RECEIVE` check in RECEIVE, filtered list in INTERROGATE). `receiver_auth` (HMAC with `TRANSFER_AUTH_KEY`) is verified *before* `PERMISSION_MAC` in the R3 block so that an unauthenticated sender cannot reach `PERM_MAC_KEY` operations with chosen permission bytes.

8.3 Side-Channel Attack Mitigations

Countermeasure	Details
Constant-time comparison	`secure_compare()` uses an XOR accumulator over the full buffer. No early exit. Used for all HMAC tag and GCM tag comparisons.
TRNG jitter (32 768 steps)	`random_delay()` reads *two* TRNG bytes: a coarse byte producing 0–≈4 ms jitter and a fine byte adding 0–≈1 ms, yielding $128 \times 256 = 32\,768$ distinct delay values. It is called inside `hmac_sha256()` before every `wc_HmacSetKey` and inside `aes_gcm_encrypt`/`decrypt` before every key load, so all HMAC and AES call sites are covered automatically.
Challenge tweak (chosen-input CPA)	`CHALLENGE_TWEAK` is XOR'd with every handshake and interrogate nonce before it enters `hmac_sha256()`. The attacker controls the wire nonce but cannot predict the actual HMAC first-block input, removing the chosen-input advantage required for CPA against `TRANSFER_AUTH_KEY`.

8.4 Memory Safety

- All packet sizes are validated against known struct sizes at entry.
- `slot` $<$ `MAX_FILE_COUNT` enforced before any flash access.
- `contents_len` $\leq$ `MAX_CONTENTS_SIZE` enforced at WRITE and RECEIVE.
- Name fields: printable ASCII only (`0x20`–`0x7E`), at least one character, null-terminated, bounded by `MAX_NAME_SIZE`.
- Permission boolean bytes in received proofs: validated to be exactly `0x00` or `0x01` by `validate_bool()`.
- `perm_count` $\leq$ `MAX_PERMS` enforced before any permission iteration.
- Cross-packet length check in RECEIVE: packet length must equal `FILE_DATA_HEADER_SIZE + contents_len`.
- `n_files` in INTERROGATE R2 is bounds-checked against the received `list_data_len` before forwarding to the host.
- `secure_zero()` called on all sensitive stack buffers before return.

8.5 Stack Canary

The firmware is compiled with `-fstack-protector-strong`. The canary (`__stack_chk_guard`) is seeded from the hardware TRNG on boot: the low byte is forced to `0x00` so any upward string-copy overwrite corrupts the canary before reaching the return address, and the upper 24 bits are random TRNG bytes to defeat non-string overwrites. `__stack_chk_fail()` delegates immediately to `security_halt()`.

9 Attack Mitigation Summary

Attack	Mitigation
Brute-force PIN via serial	5 s delay per failure; 2000 ms UART timeout per byte; 16^6 space takes ≈2 years online.
CPA against AES keys	AESADV hardware; `random_delay()` (0–≈5 ms, 32 768 steps) before every key load; peripheral reset after each call; four independent keys so one compromise reveals nothing about others.
CPA against HMAC keys	`random_delay()` inside every `hmac_sha256()` call; `CHALLENGE_TWEAK` XOR breaks chosen-input advantage on `TRANSFER_AUTH_KEY`; `PERM_MAC_KEY` never enters runtime hardware; `PIN_KEY` is per-device.
Offline PIN brute-force	16^6 HMAC evaluations required even with `PIN_KEY` and `PIN_HMAC` in hand; per-device key limits blast radius.
Single-glitch bypass (PIN, permission, GCM tag)	Double-pass evaluation on all three; `SECURE_CHECK_FAILED` complement-canary hardening on each branch; volatile double-check of GCM hardware return value and tag booleans (FI2/FI3); mismatch calls `security_halt()`.
Unauthenticated sender choosing permission bytes	`receiver_auth` verified *before* `PERMISSION_MAC` in R3; unauthenticated sender cannot reach `PERM_MAC_KEY`.
Replay of previous transfer	Fresh TRNG challenge per session; challenges bound into Transfer AAD.
Rogue device / permission forgery	Mutual HMAC challenge-response; `PERMISSION_MAC` = HMAC(`PERM_MAC_KEY`, ...) never recoverable via runtime CPA; GCM tag authenticates file content.
Metadata manipulation (group, slot, name)	All three fields bound into both storage AAD and transfer AAD; GCM tag fails on any alteration.
TOCTOU race on READ	`group_id` re-verified after decryption; mismatch returns error.
Stack smash	`-fstack-protector-strong`; TRNG-seeded canary with embedded null byte defeats string-copy overwrites.